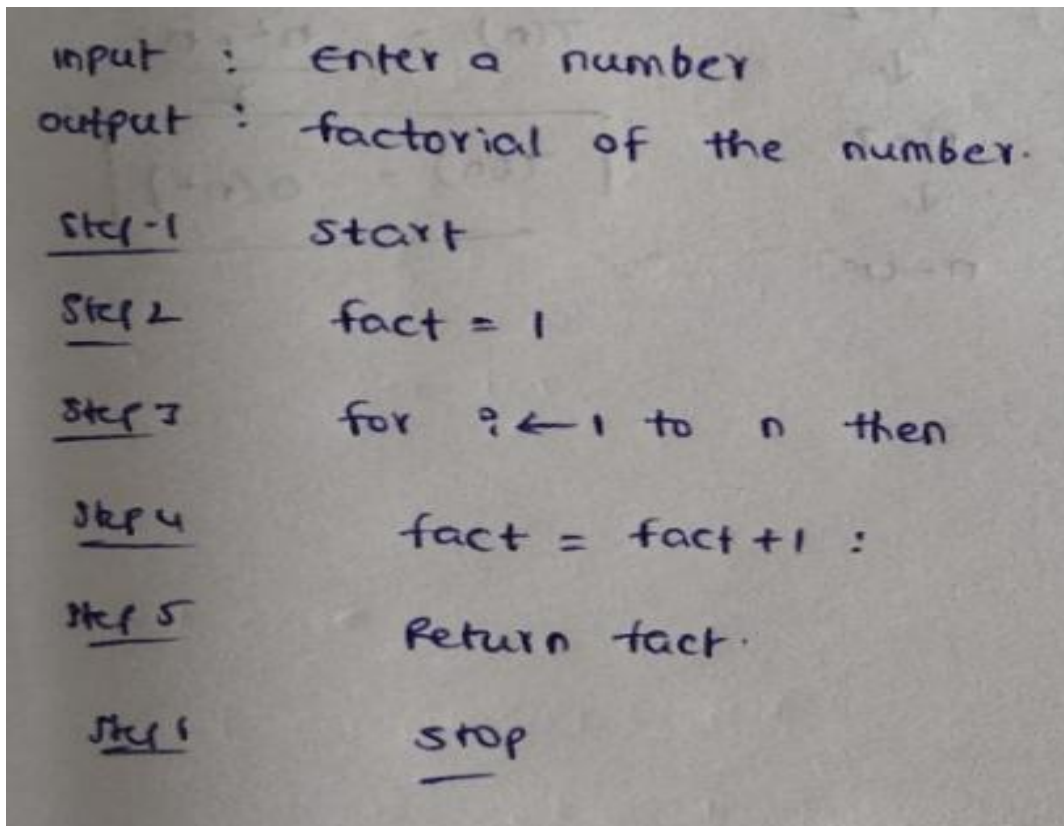


PRACTICAL-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial Using Iterative Method

ALGORITHM:



```
input : Enter a number
output : factorial of the number.

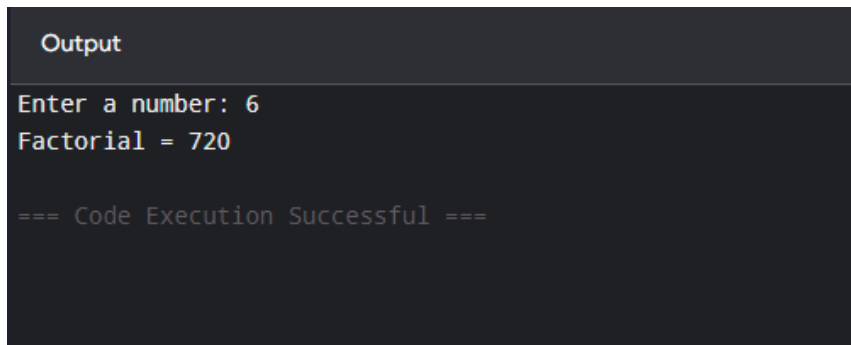
Step 1 start
Step 2 fact = 1
Step 3 for i ← 1 to n then
Step 4     fact = fact + 1 :
Step 5     Return fact.
Step 6 stop
```

CODE:

```
#include <iostream>
using namespace std;
int factorial(int n)
{
    int fact = 1;
    for(int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    return fact;
}
```

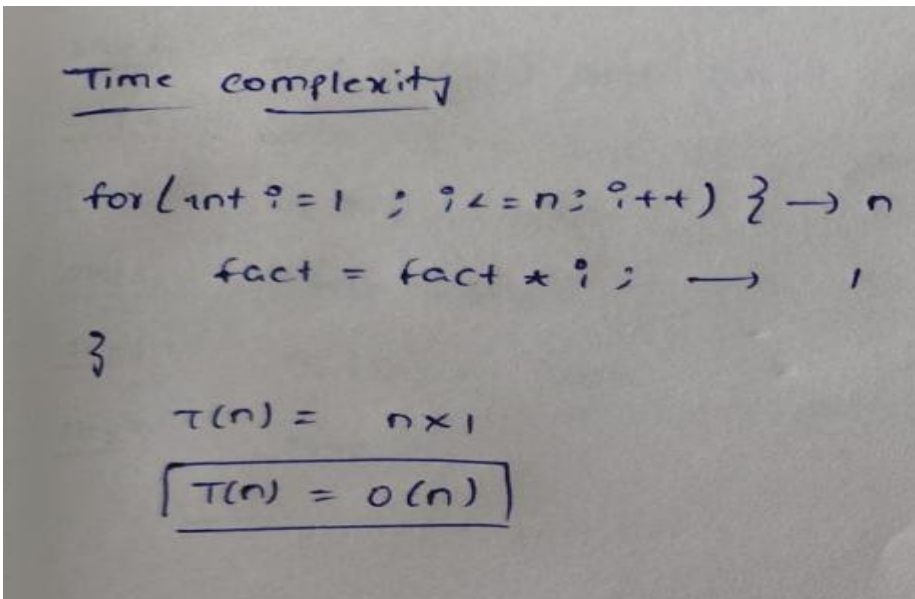
```
}  
  
int main()  
{  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
  
    cout << "Factorial = " << factorial(n);  
  
    return 0;  
}
```

OUTPUT:



```
Output  
Enter a number: 6  
Factorial = 720  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



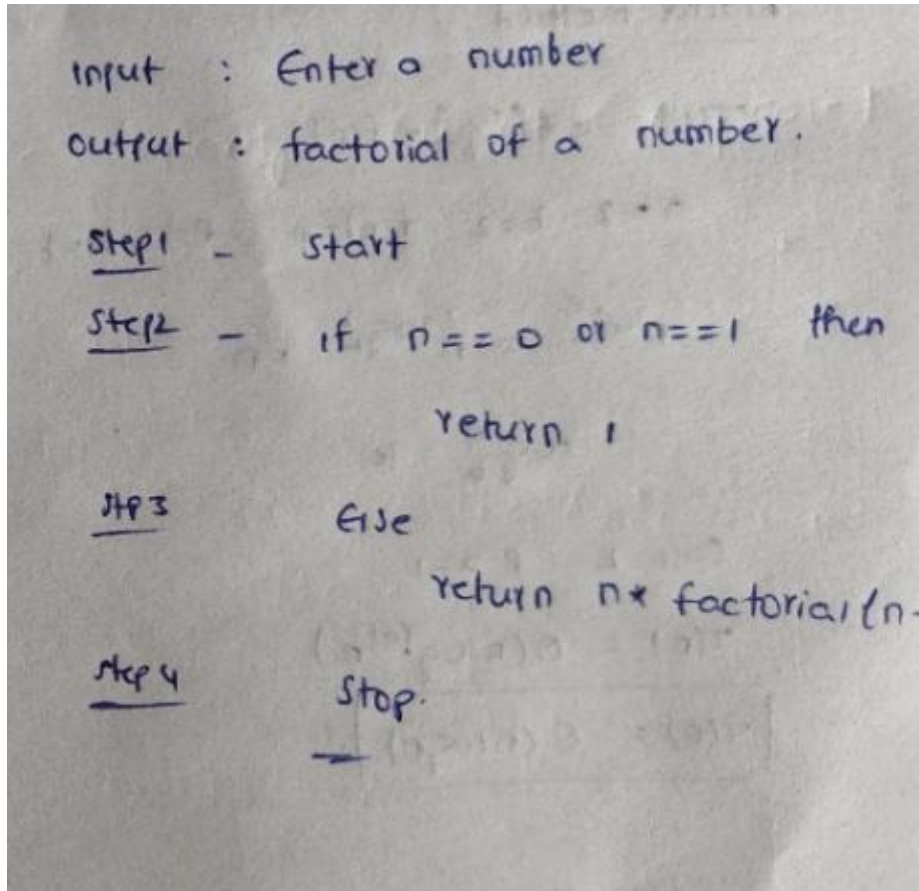
Time complexity

for (int i = 1 ; i <= n ; i++) } $\rightarrow n$
 fact = fact * i ; $\rightarrow 1$
}

$T(n) = n \times 1$
 $T(n) = O(n)$

Factorial Using Recursive Method:

Algorithm:

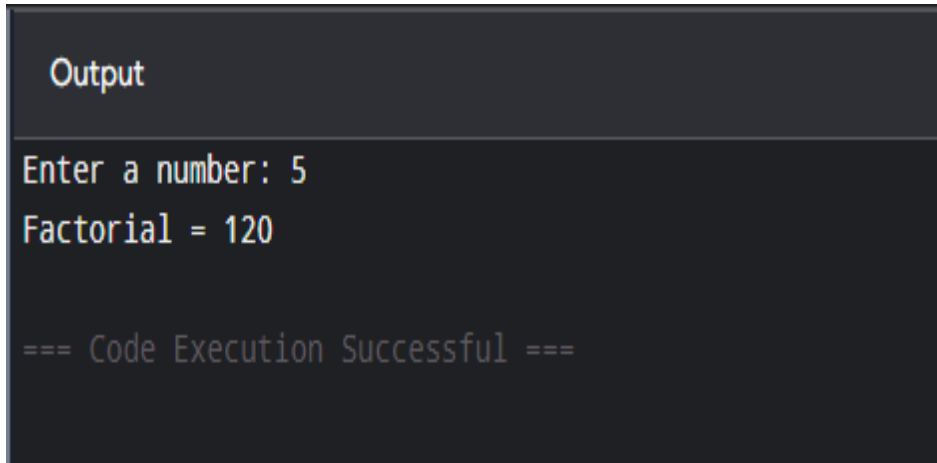


CODE:

```
#include <iostream>
using namespace std;
int factorial(int n)
{
    if(n == 0 || n == 1)
    {
        return 1;
    }
}
```

```
    return n * factorial(n - 1);  
}  
int main()  
{  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial = " << factorial(n);  
    return 0;  
}
```

OUTPUT:



The screenshot shows a dark-themed terminal window. At the top, the word "Output" is displayed in a light blue font. Below it, the program's output is shown in a light green monospaced font: "Enter a number: 5" followed by "Factorial = 120". At the bottom, a status message "=== Code Execution Successful ===" is displayed in a light blue font.

TIME COMPLEXITY:

Time complexity

int factorial (int n) { $\rightarrow T(n)$

if (n==0 || n==1) { $\rightarrow n$

return 1; $\rightarrow 1$

}
else {

return n * factorial (n-1) $\rightarrow n-1$

}

}

$T(n) = (n, n-1)$

$T(n) = \max (n, n-1)$

$T(n) = O(n)$

CONCLUSION:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.